# Join Implementations

# Review of semantics

Several similar operations:

- Cross join
- Inner join
- Variations on what to do with dangling tuples

# Overview of implementations

Three main strategies:

- Nested joins + 2 variants
- Sort-merge joins + 1 variant
- Hash joins

Notation:

- Join(R, S) on R.a = S.a
- T(R) = #tuples in R
- B(R) = #blocks in R

# Nested loop join

```
For r in R:
    For s in S:
        If r.a == s.a, yield (r,s)
```

```
For s in S:
    For r in R:
        If r.a == s.a, yield (r,s)
```

# Block nested loop join – I/O aware

Let B = #blocks in memory

```
For each group of B-2 blocks br of R:
    For block bs of S:
        For r in br:
            For s in bs:
                If r.a == s.a, yield (r,s)
```

# Index nested loop join

If S is indexed on S.a.

For r in R:
    For s in $\text{index}_{S.a}$(r.a):
        yield (r,s)

# Sort-merge join

1. Sort R and S each on attribute a.
   - Use *external merge sort* (disk-based).
   - But, R and/or S might already be sorted.
2. "Merge" R and S.
   - Linear scan R.
   - Linear scan S, backing up on duplicates of R.a.

Merge example:

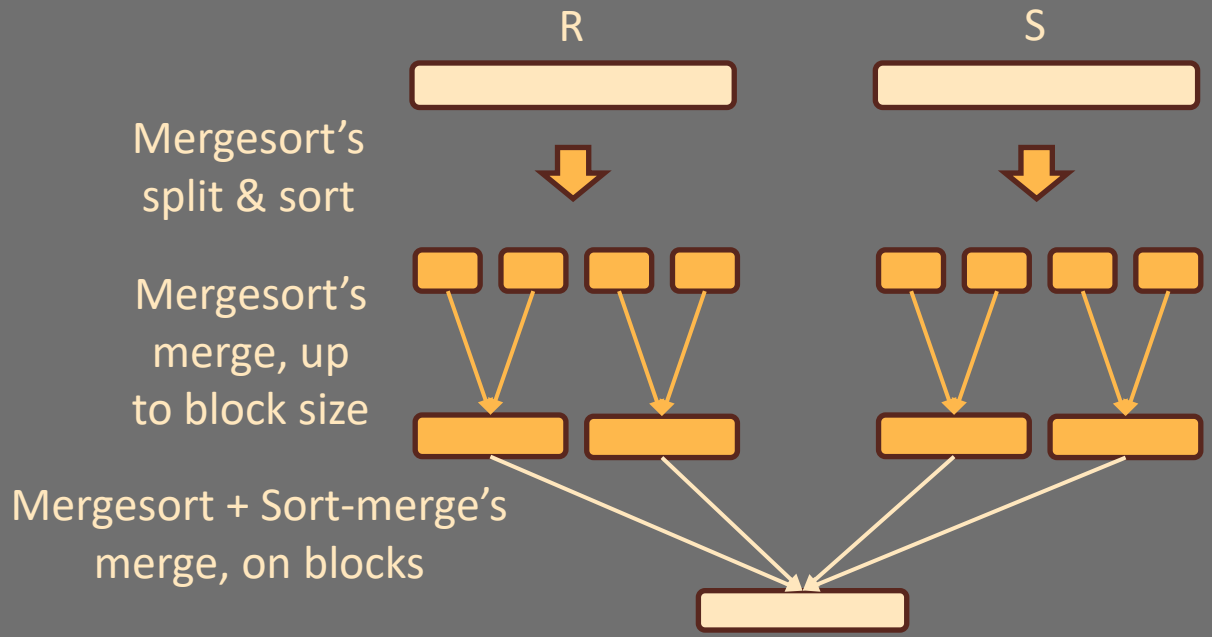
R: (1,a) (2,b) (2,c) (3,d)

S: (1,v) (2,w) (2,x) (3,y) (4,z)

Not too sensitive to skew.
Leaves data sorted!

# Sort-merge join – I/O aware

Integrate the mergesort's merging with the "merge" of R,S.

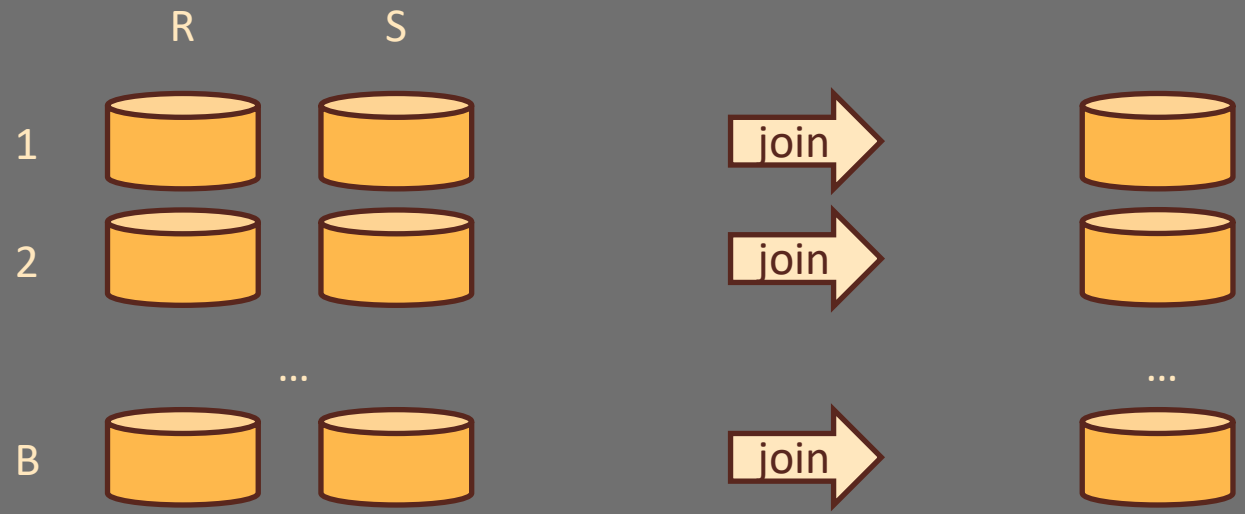


# Hash-join

Use hashing to decompose one large join into many small joins.

Step 1: Hash R, S each on same hash function into buckets.

Step 2: Join pairs of buckets. Hash-join recursively or block nested join.




Highly parallelizable